Open in app ↗ ı up Sign in

Mediu

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Striim · Follow publication

Duckdb + Database + Python + Sql + Software Development +

Beyond Materialized Views: Using DuckDB for In-Process Columnar Caching

11 min read · Apr 2, 2025



John Kutay Follow

Listen Share

Written by John Kutay (Director of Product & Engineering at Striim) and Kim Dang (Senior Software Engineer at Striim)

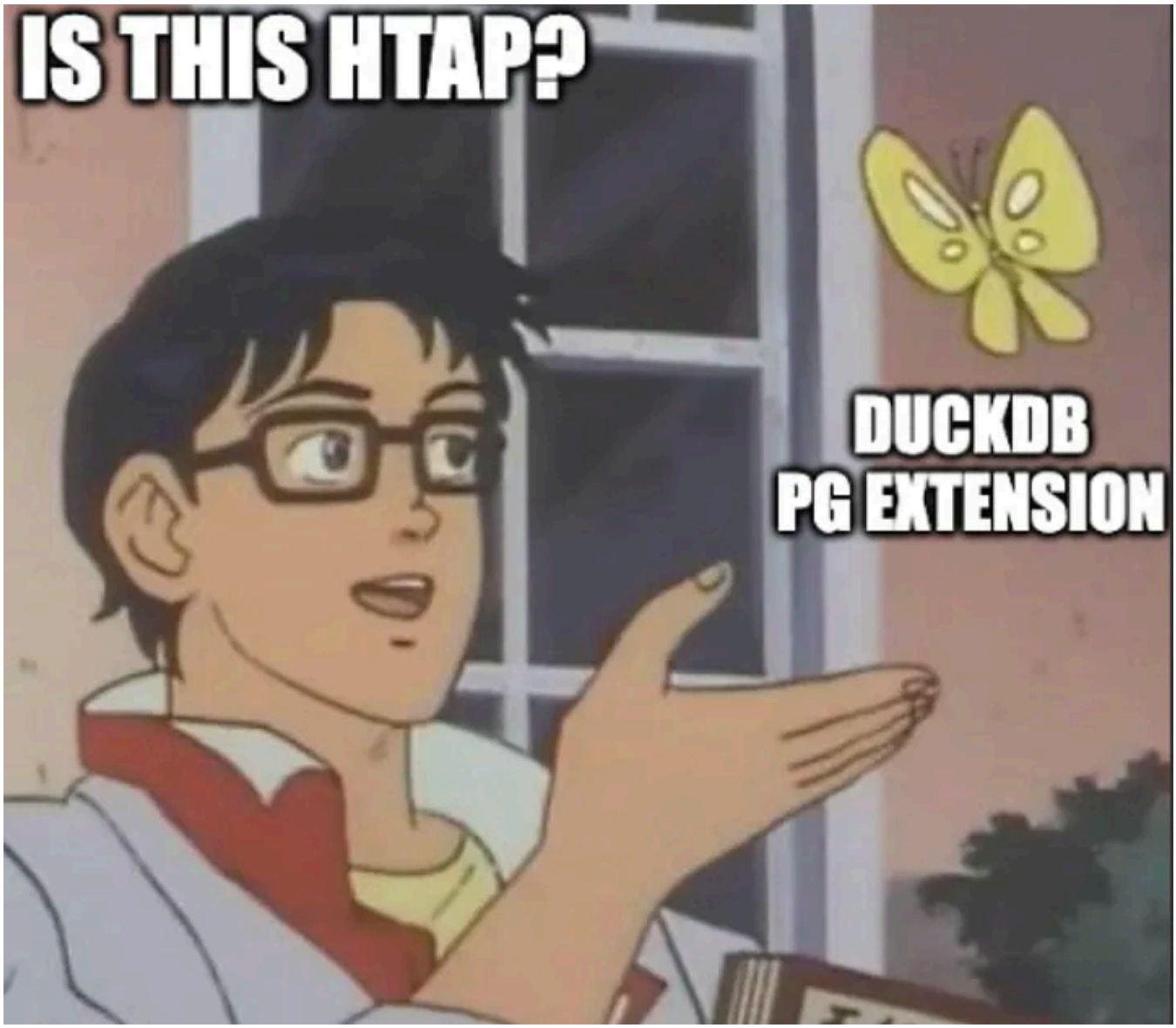
In this post we will talk about using DuckDB as the operational analytics store for the control plane of Striim Developer — a serverless Stream Processing and Change Data Capture service. By moving analytical queries from PostgreSQL to an in-process cache in DuckDB, we measured 5–10x performance improvement with zero additional infrastructure or cost.

In this post, we explore how DuckDB was integrated as the analytical backend for the control plane of **Striim Developer** — a serverless Stream Processing and Change Data Capture (CDC) service. The Striim Developer Control Plane handles critical tasks such as managing tenant assignments, monitoring free-tier usage thresholds, orchestrating asynchronous jobs, and triggering operational alerts. Initially, this control plane relied on direct queries to PostgreSQL, which serves as the metadata repository. However, this design presented challenges in maintaining performance and meeting SLAs without inflating the cost of offering a free-tier service.

For example, when usage exceeds capacity for an individual tenant, we need to alert and orchestrate a response immediately. Yet, our previous implementation — based

on periodic analytical jobs — was constrained to 15–20-minute intervals, requiring PostgreSQL This latency po less platform that necessitates near real-time responses.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



After evaluating alternatives, we adopted DuckDB as an in-process, columnar caching layer. This decision was driven by DuckDB's balance of performance, simplicity, and minimal overhead, addressing our need for high-frequency analytics while offloading PostgreSQL. In this post, we detail the challenges, implementation, and benefits of this architecture and discuss whether this approach qualifies as Hybrid Transactional Analytical Processing (spoiler: it's not, and I'll explain why).

The Challenge: Optimizing Operational Analytical Workloads

The Striim Developer control plane frequently queries PostgreSQL to perform:

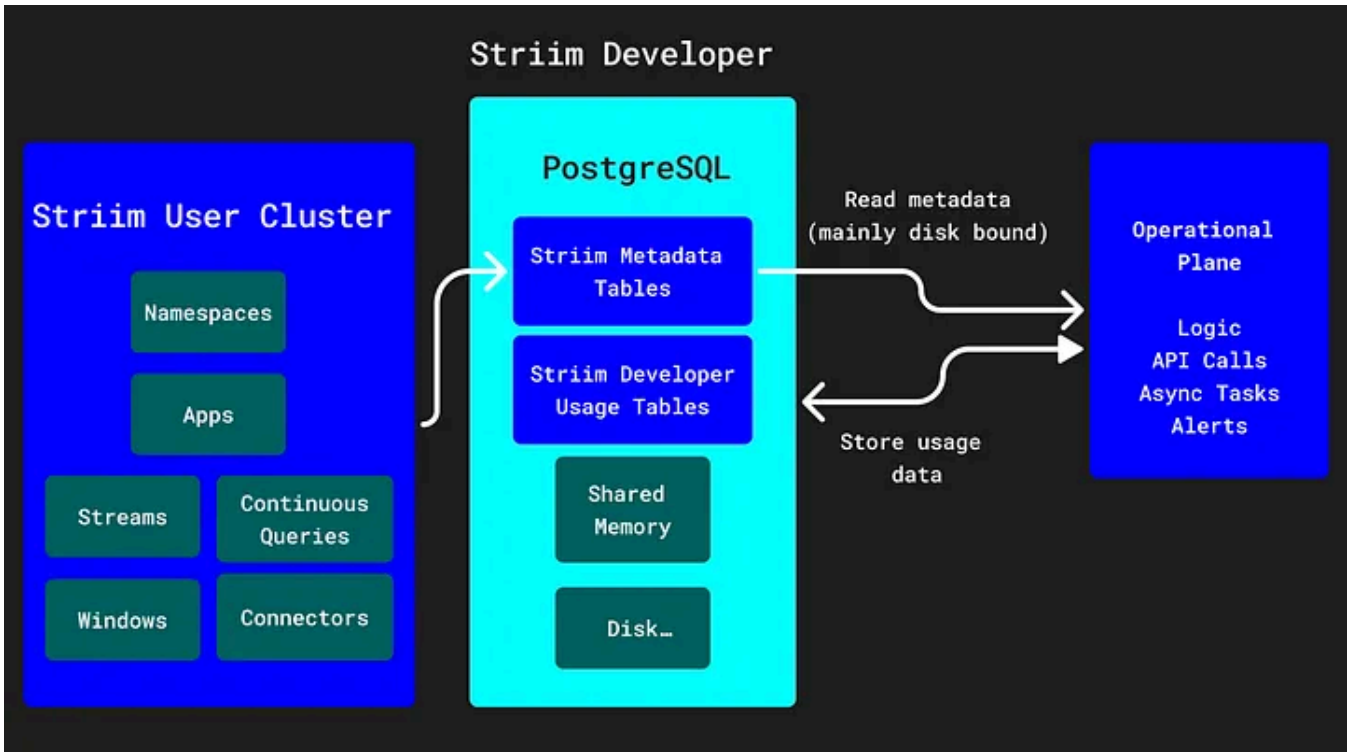
- Aggregated usage calculations

- Threshold monitoring for user and tenant consumption
- Operat

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

- Asynchronous calls to cloud services that trigger workflows (in-product messages, cloud resource management)

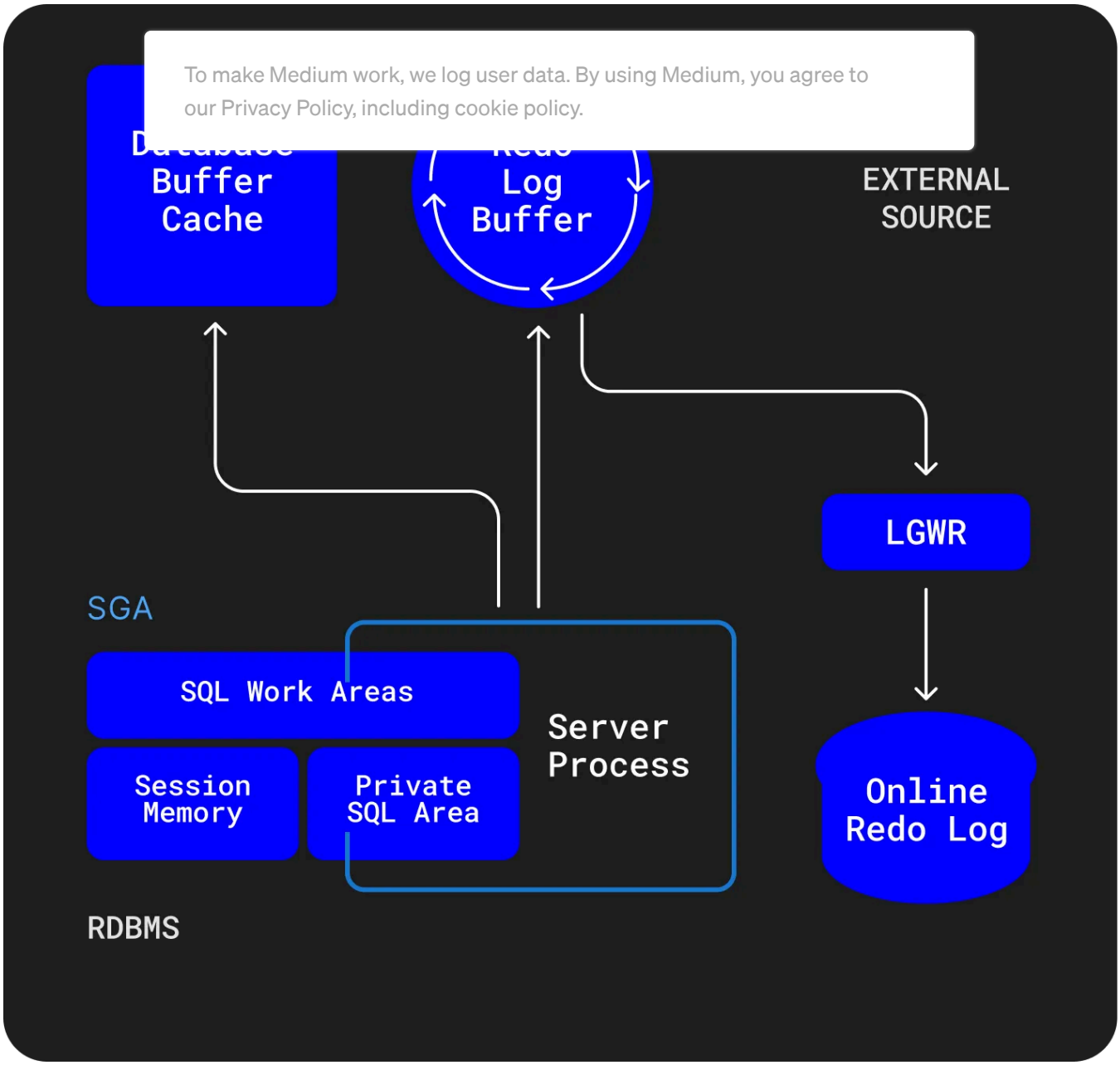
These queries are analytical and involve scanning and joining usage data normalized across many tables, leading to slower execution times and increased load on our PostgreSQL instance. However our ability to frequently run this operation was bottlenecked by disk-bound operations to query PostgreSQL. We require these analytical jobs to run as frequently as possible to execute to respond to scenarios like resource over-utilization or exceeding usage thresholds.



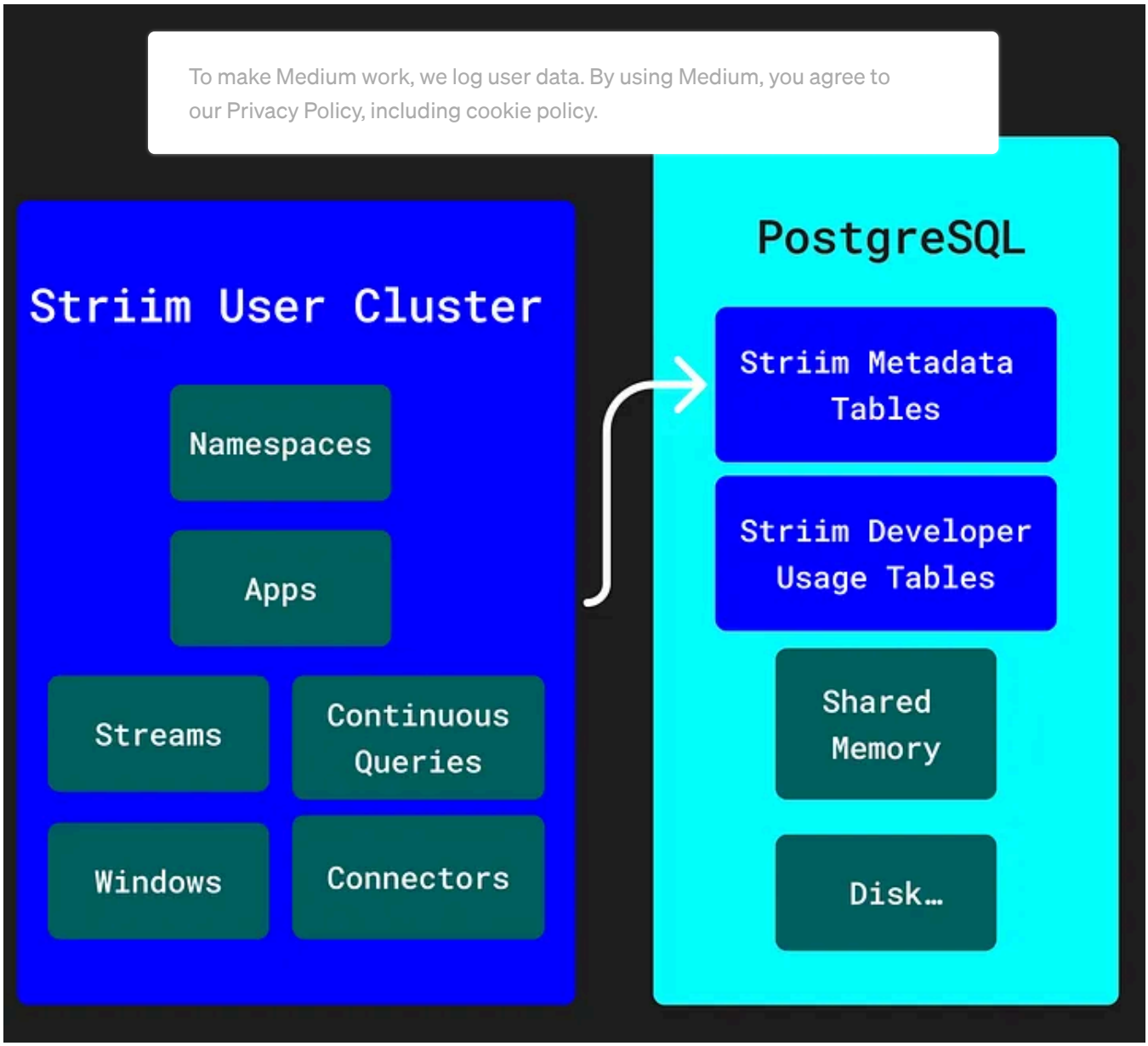
Database Application Behind Striim Developer

A Word About Buffer Caches

Before we went down this rabbit hole, we of course considered the stark reality that database buffer caches exist. And our AWS-managed database has some higher level interfaces to manage the buffer cache that adds some convenience for DBAs.



PostgreSQL's buffer cache can improve query performance by storing frequently accessed data pages in memory, minimizing disk I/O. While tuning shared_buffers and leveraging tools like pg_prewarm could help keep critical data in cache, this approach is inherently reactive. The cache is populated dynamically based on usage patterns, which means frequent refreshes or large scans may still involve costly disk reads. Additionally, tuning buffer sizes in a multi-tenant environment adds complexity and can strain memory resources shared with other PostgreSQL processes.



It's also important to note that the Striim application is the software most frequently accessing the PostgreSQL metadata repository with frequent, write-heavy operations that naturally dominate the PostgreSQL buffer cache. It handles critical transactional operations like updating metering metrics and managing the Striim component DDL as streaming applications are implemented by users. This competition for cache space means analytical queries, such as aggregations on usage data, are less likely to benefit from cached pages, resulting in increased disk I/O and slower performance. Given this dynamic, relying solely on PostgreSQL's buffer cache to optimize both OLTP and OLAP workloads would have been inefficient, reinforcing our decision to offload analytical queries to DuckDB. By decoupling these workloads, we ensured that transactional operations in PostgreSQL remain fast while analytical workloads benefit from DuckDB's optimized in-memory columnar engine.

Materialized views provide precomputed query results and might seem like a fit for aggregatin ry minute for usage d with REFRESH CONCURRENTLY, the performance cost grows as data scales. Managing MVs for our multi-tenant architecture, with differing refresh cadences for namespaces, applications, and usage data, would have added operational burden.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Why Not Materialized Views?

Materialized views in PostgreSQL were an option we considered for caching precomputed results. However we had to address the following variables:

1. Analytical Queries: Our queries involve dynamic aggregations, joins and filters on frequently changing usage data, which are cumbersome to maintain with materialized views. Our team prefers working with imperative languages for cache maintenance logic, thus maintaining the views in PostgreSQL was not ideal.

2. Infrastructure Overhead: Refreshing materialized views for frequently updated data (e.g., usage metrics) every minute would increase PostgreSQL's load rather than alleviate it. PostgreSQL read replicas would add more complexity and infrastructure overhead without obvious performance benefits.

3. Limited Flexibility: PostgreSQL materialized views speed up complex queries but require manual refreshes, lacking incremental updates for frequent changes.

The ideal solution for us would require no additional infrastructure (replicas, database cores, storage) at the current size of the workload while allowing us to tightly control the cache maintenance logic in Python — the language we use in our Control Plane.

Our Solution: DuckDB as a OLAP Cache with PostgreSQL Extension

DuckDB, a lightweight, in-process SQL analytics engine, provided the perfect middle ground between raw PostgreSQL queries and materialized views. Here's why:

- High Performance for Analytical Workloads: DuckDB is automatically optimized for analytical and columnar processing, offering built-in performance gains for our use case.

- Simple Integration with Python: Its seamless embedded Python API made it easy to infrast

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

- Dynamic Caching: We cache infrequently changing data (e.g., users, tenants) every 24 hours and refresh frequently changing data (e.g., usage metrics) every minute. DuckDB makes this easy to manage with minimal code.
- PostgreSQL Extension: DuckDB's native extension lets us execute PostgreSQL queries to fetch data within the DuckDB runtime

DuckDB's in-process instantiation made this insanely easy to implement in the control plane application...

```
class Cache:
    def __init__(self, cache_name: str) -> None:
        self.cache_name = cache_name
        self.conn = duckdb.connect(database=':memory:')
```

Instantiating DuckDB In-Memory

Implementation Overview

Data Refresh Pipeline:

- Daily Refresh: Less frequently updated metadata loaded into DuckDB every 24 hours.
- Minute-Level Refresh: Frequently queried usage metrics are updated every minute.

Query Execution:

- Operational tasks and alerts query the DuckDB cache for aggregated results.
- Async cloud operations work with fresh, fast data.
- PostgreSQL remains the source of truth, used only for updates and less frequent transactional queries.

The architecture ensures that DuckDB serves as a high-speed cache for read-heavy operations, while PostgreSQL is offloaded from expensive analytical workloads.

Performance Benchmarks

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Our perfor ined by integrating erates on 4 vCPUs and 7 GB of RAM, emphasizing efficiency in constrained environments. Below are the results from the latest test runs:

Results with DuckDB Caching:

- Throughput of jobs: Improved from 3.95 to 11.71 tasks/sec (cached + PostgreSQL replication) and up to 11.71 tasks/sec (full analytical query caching).
- Average Latency: Consistent at 0.19–0.2 seconds per task.
- Memory Usage: Stabilized at ~141 MB (cached namespace scenario) and ~120 MB (full analytical query caching).
- Execution Time: Reduced to as low as .8 seconds per job.

To ensure the accuracy and consistency of our benchmarks, we conducted multiple runs for each configuration — No Caching, Caching data with SQLAlchemy into native python data structures, and Caching with DuckDB — with explicit cache warming captured by mocking multiple runs from cold to hot cache scenarios. By allowing the cache to warm up, we ensured that frequently accessed data was preloaded, highlighting the true impact of caching mechanisms on throughput, execution time, and memory stabilization under high-concurrency workloads. This approach provided a clear view of how caching progressively improves performance in real-world scenarios and DuckDB provides built-in high performance analytical queries.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Metric			vith
Throughput (tasks/ sec)	14.37 → 0.38 **Throughput decreases**	3.92 → 7.86	2.73 → 11.71 **Higher throughput on each run**
Average Latency (sec/task)	0.88	0.19	0.19
Memory Usage (MB)	76.68 - 77.11	154.39 → 154.79	151.15 → 120.28 **Less memory usage on each run**
Job Execution Time (seconds)	303.95 - 606.23	2.04 → 1.02 Faster execution on each run	1.20 → 0.82 **Faster execution on each run**

Key Takeaways:

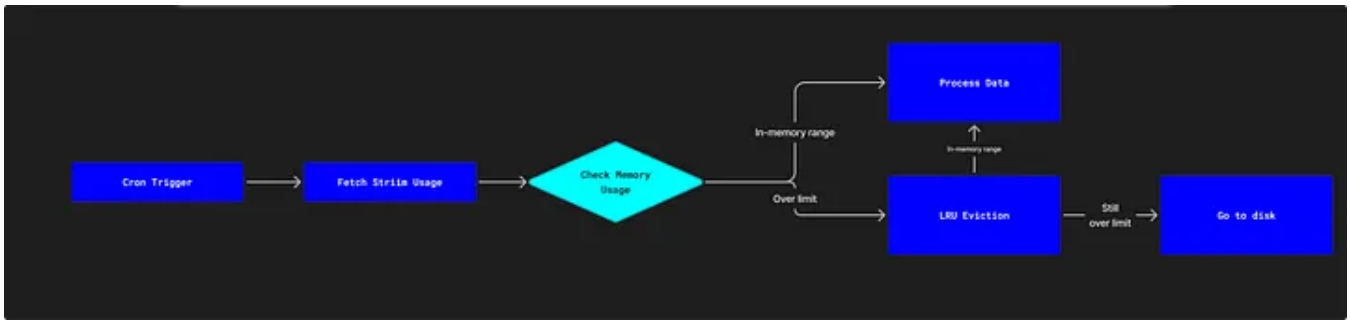
1. 5–10x Execution Time Reduction: Analytical workloads experienced dramatic speed-ups without impacting transactional performance.
2. High-Throughput Performance: Throughput improved significantly for tasks, maintaining low latency.
3. Zero Additional Infrastructure Costs: DuckDB runs in-process, avoiding the need for more servers or storage.
4. DuckDB-based caching with PostgreSQL extension was faster than native python solution

This setup is designed for the price-performance balance that makes our Striim Developer offering scalable while remaining cost-effective. While scaling CPU and RAM might have solved the problem, this approach aligns with our mission to deliver generous free-tier usage without compromising on performance.

Memory management

While performance improved by 5–10x, we need to account for the increased memory us

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



Memory Management & Cache Eviction Strategy

We will monitor and log our cache hit rate over time while managing a simple heuristic based memory management process. Rather than using fixed memory limits, we designed a percentage-based approach where the cache can use up to 75% of system RAM total, with individual caches limited to 25%. This provides flexibility across different deployment environments while preventing memory exhaustion. We designed an LRU (Least Recently Used) eviction strategy that automatically removes older entries when memory limits are approached, along with a fallback mechanism to persist data to disk if needed. The cache monitors system memory usage and can trigger eviction or disk fallback (DuckDB PG extension queries) automatically.

Why DuckDB Cache is Superior

Compared to materialized views or other caching alternatives (e.g., Redis, Native Python Data Structures), DuckDB offers:

Get John Kutay's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Performance Gains Without Complexity:

- No additional servers, clusters, or infrastructure were required
- No nee

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

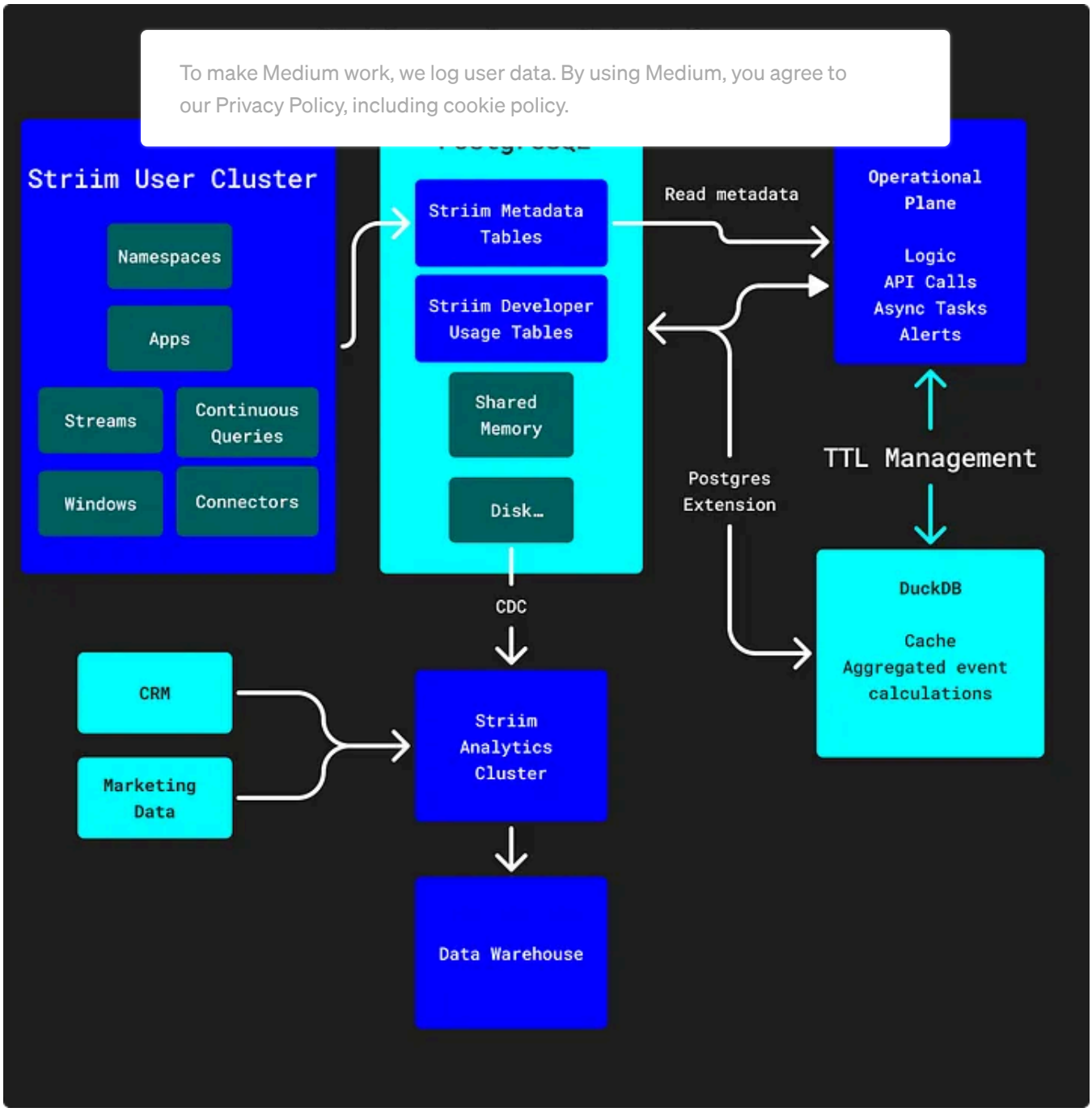
- All cache maintenance logic implemented in python

Flexibility and Simplicity:

- Easy to define and refresh datasets based on different refresh cadences (e.g., 24 hours for static data, 1 minute for dynamic data).
- Dynamic and on-the-fly SQL queries supported without materialized view limitations.
- SQL interface for aggregations and analytical queries

Cost-Effective Scaling:

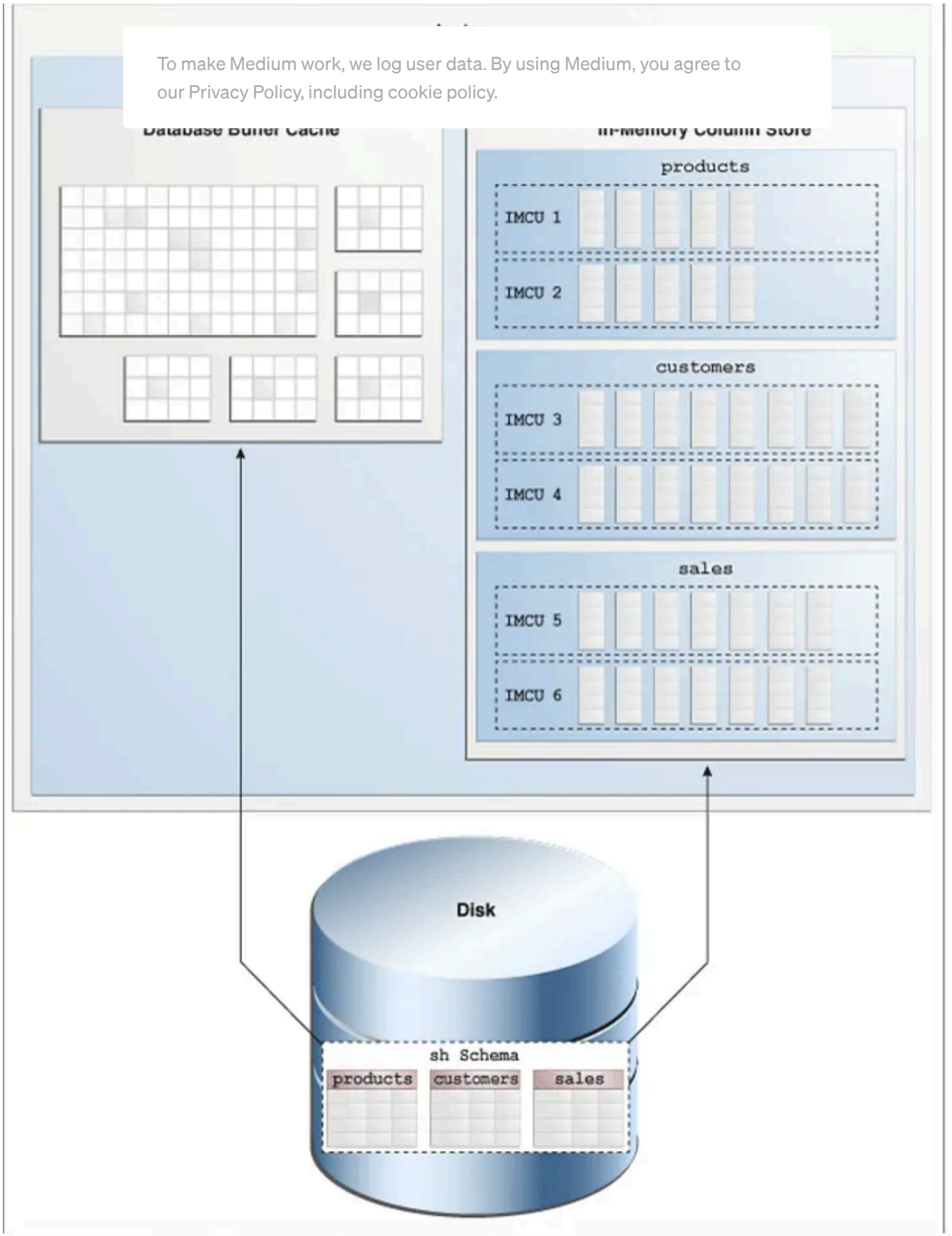
- DuckDB runs within the existing Python application with optimized query execution and memory management, avoiding the need for external caching systems or compute resources at the size of the current workload.



So... is this HTAP?

While this approach leverages DuckDB's fast analytical performance and offers many benefits that you would get from an HTAP system, it doesn't qualify as full HTAP in my mind. HTAP systems unify OLTP and OLAP workloads within a single platform, providing real-time analytics on live transactional data, an example being Oracle's embedded column store. In our case, PostgreSQL handles transactions, while DuckDB serves as a pluggable, in-memory column store for read-heavy analytical queries. DuckDB PostgreSQL Extension and our controller logic glues OLTP and OLAP together by copying the data. This separation allows us to periodically refresh cached data without adding complexity or infrastructure.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



Description of "Figure 2-4 IM Column Store"

Source: Oracle

Hybrid transactional/analytical processing (HTAP) architectures integrate OLTP (transactional) and OLAP (analytical) workloads in a single system, eliminating the

need for ETL pipelines, reducing data duplication, and offering real-time consistenc of HTAP, maintainir ormat (optimized for OLTP) in the buffer cache and in a columnar format (optimized for OLAP) in memory. Oracle ensures transactional consistency by synchronizing updates between the row and column formats using metadata and transaction journals. This allows analytic and transactional queries to run on the same dataset seamlessly, without duplication or delays.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Our DuckDB PostgreSQL extension approach, while not HTAP, delivers basic hybrid OLAP functionality by enabling analytical queries to run in a high-performance columnar format, separate from the OLTP workload in PostgreSQL. Unlike Oracle's dual-format architecture, this solution doesn't maintain real-time consistency or simultaneous access to row and columnar data. However, it provides us with a lightweight, pluggable option for analytical workloads with minimal setup. Given these are analytical operational queries, we can live without transactional consistency. DuckDB's columnar processing and in-memory execution make it ideal for scenarios where transactional workloads can tolerate slightly delayed analytics without requiring a full HTAP system.

DuckDB works great as a modular, lightweight columnar engine that complements any database, delivering OLAP-like performance without needing a full HTAP database.

Scaling Considerations

While our DuckDB caching layer delivers excellent performance for individual clusters, you might wonder how we'll scale as Striim Developer adoption grows. The answer lies in horizontal partitioning — we'll distribute users across multiple independent Striim clusters, each with its own PostgreSQL database and controller instance running our DuckDB cache.

We leverage Striim Deployment Groups (DG) to create separation between logical and physical resources. A Deployment Group represents some unit of compute that a Striim pipeline can be deployed to on one or multiple nodes. That unit can be a single server, a shared pool, or a dedicated multi-node cluster of EC2 instances in AWS. Each user has an assigned DG that maps to some physical resource. The physical resources can be scaled up vertically or horizontally, but the DG's

association to these resources remains the same. When a Striim app is deployed to a DG, it will

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

A global routing layer using consistent hashing will direct users to their assigned cluster based on their tenant ID, with the ability to dynamically add new clusters as needed. This approach is particularly clean because Striim Developer users operate independently, meaning there's no cross-user data sharing or multi-tenant querying that would complicate the architecture. The beauty of this design is that our DuckDB caching layer requires no modifications — it continues to work as designed within each cluster's scope, while we achieve horizontal scale through simple user partitioning.

Conclusion

By adopting DuckDB as a caching layer for our PostgreSQL database, we achieved a substantial improvement in performance while simplifying our architecture. This approach highlights the power of embedding lightweight, high-performance solutions like DuckDB into existing systems. For anyone dealing with analytical workloads on operational data, DuckDB offers an elegant, cost-effective alternative to materialized views or external caching systems.

The best part of this? It was easy to implement thanks to DuckDB's simplicity. Zero infrastructure changes. Zero additional spend. Just some well-written Python and DuckDB PostgreSQL Extension.

These types of performance improvements allow us to provide Striim Developer as a free, serverless experience for data engineers to explore Striim's data movement and stream processing capabilities.

Duckdb | Database | Python | Sql | Software Development

stri

Follow

Published in Striim

20 followers · Last published Apr 2, 2025

Intelligent Integration with data streaming, change data capture, and AI pipelines

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



Follow

Written by John Kutay

42 followers · 11 following

Director of Engineering at Rippling

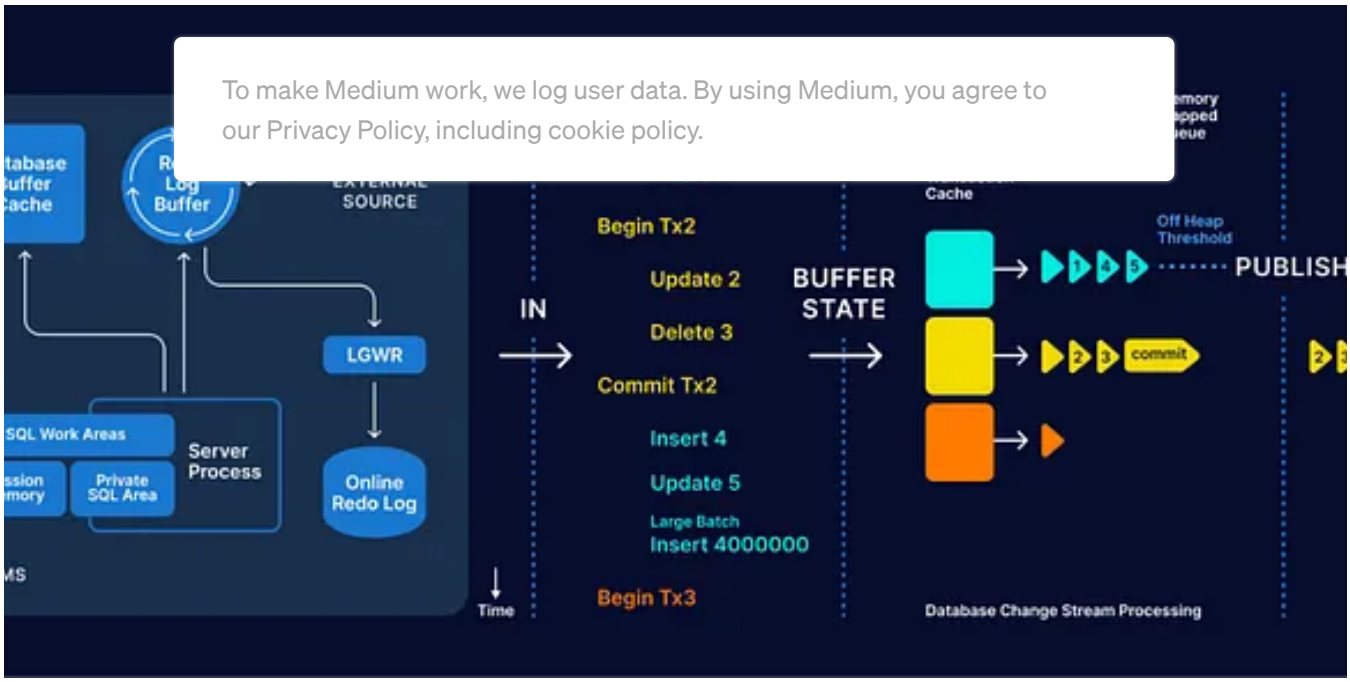
No responses yet

Write a response

What are your thoughts?

More from John Kutay and Striim

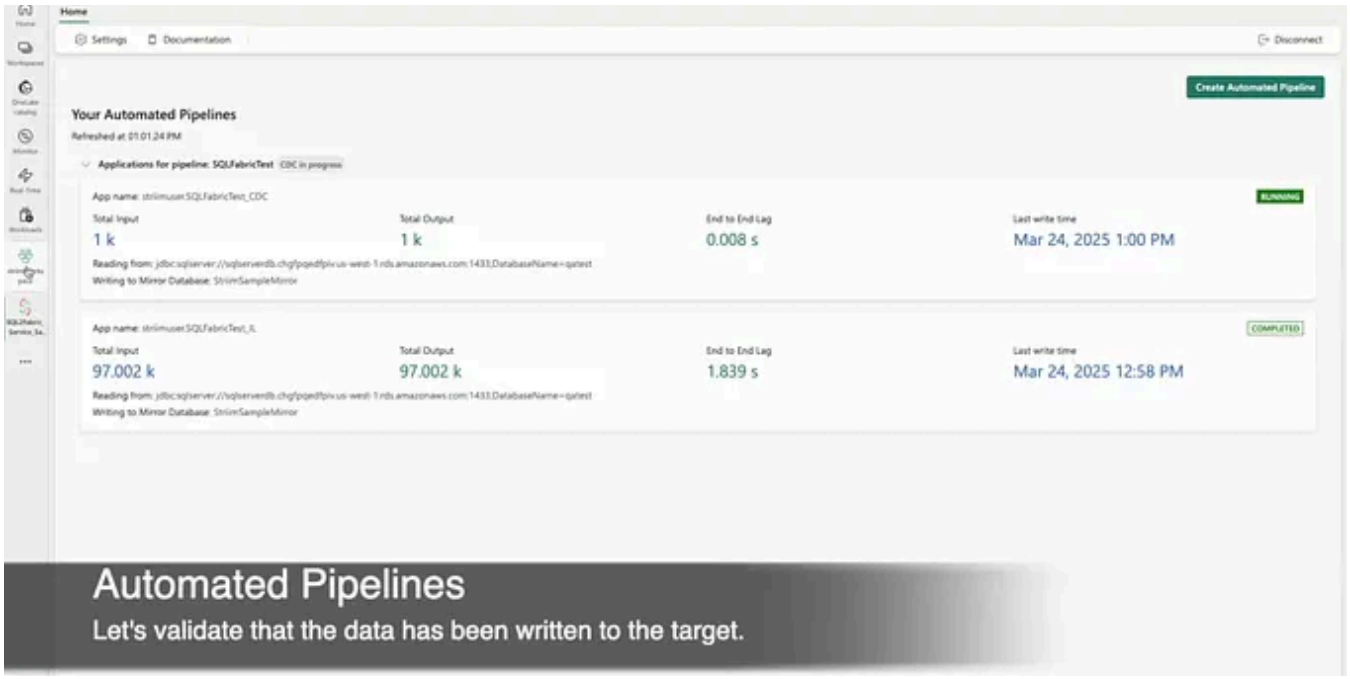
To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



In Striim by John Kutay · May 11, 2024

How Change Data Capture Works: Understanding the Impact on Databases

25

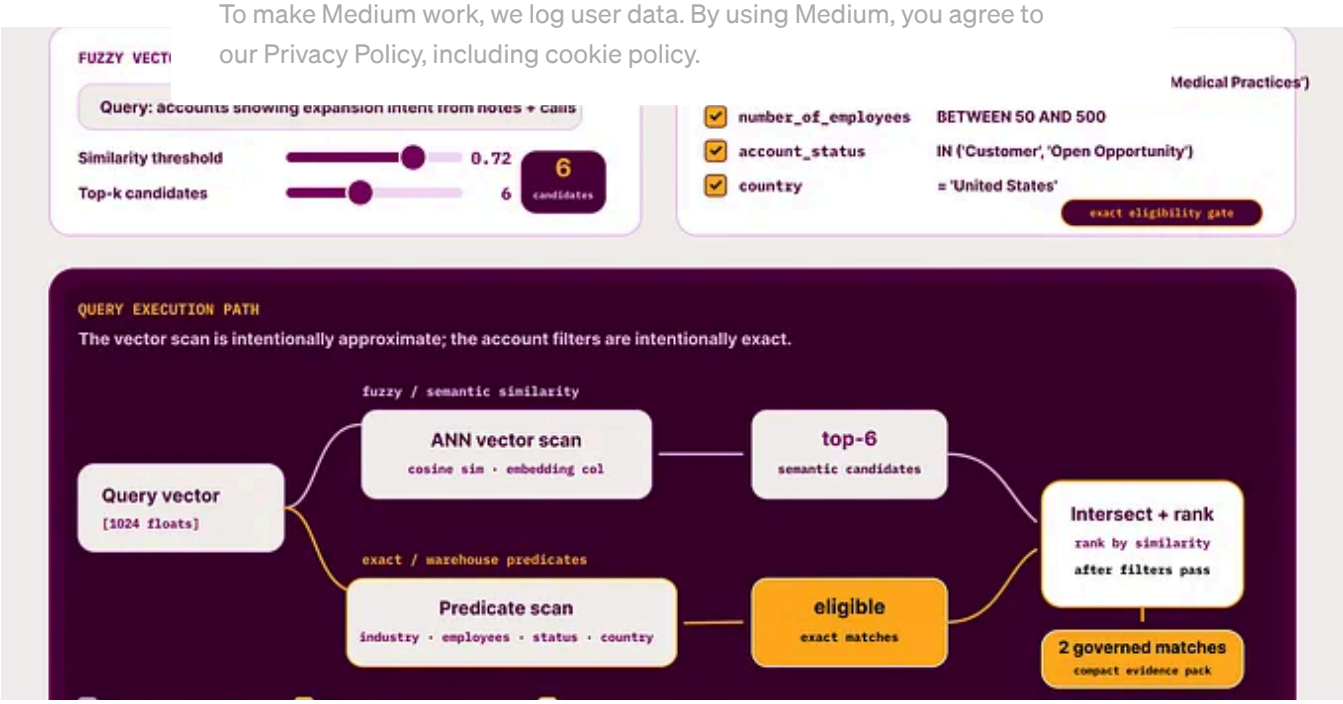


In Striim by karthikeyan G · Mar 31, 2025

Microsoft Fabric Open Mirroring—Mirror once query anywhere—Striim

This blog explores Open Mirroring and how Striim provides integration for it. It also discusses various use cases for leveraging platforms...

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



John Kutay · Jun 13

Building an AI Database for Agentic GTM Operations

How our growth engineering team rebuilt its go-to-market data foundation on a lakehouse — and turned it into a real-time intelligence layer...

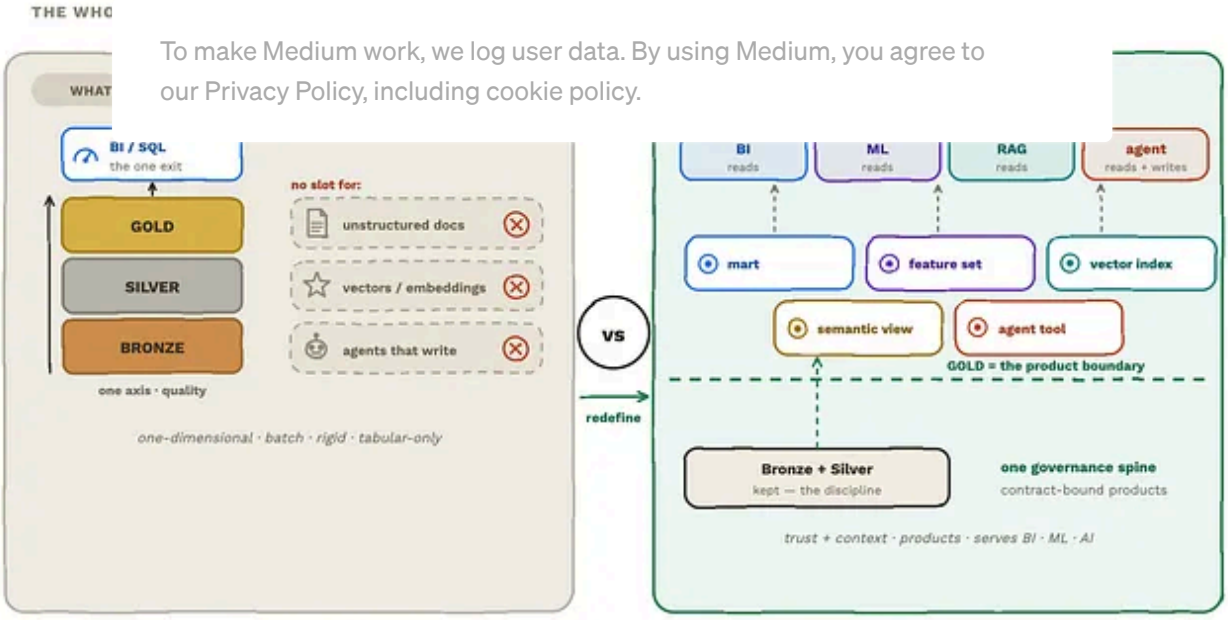
10 2

See all from John Kutay

See all from Striim

Recommended from Medium

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

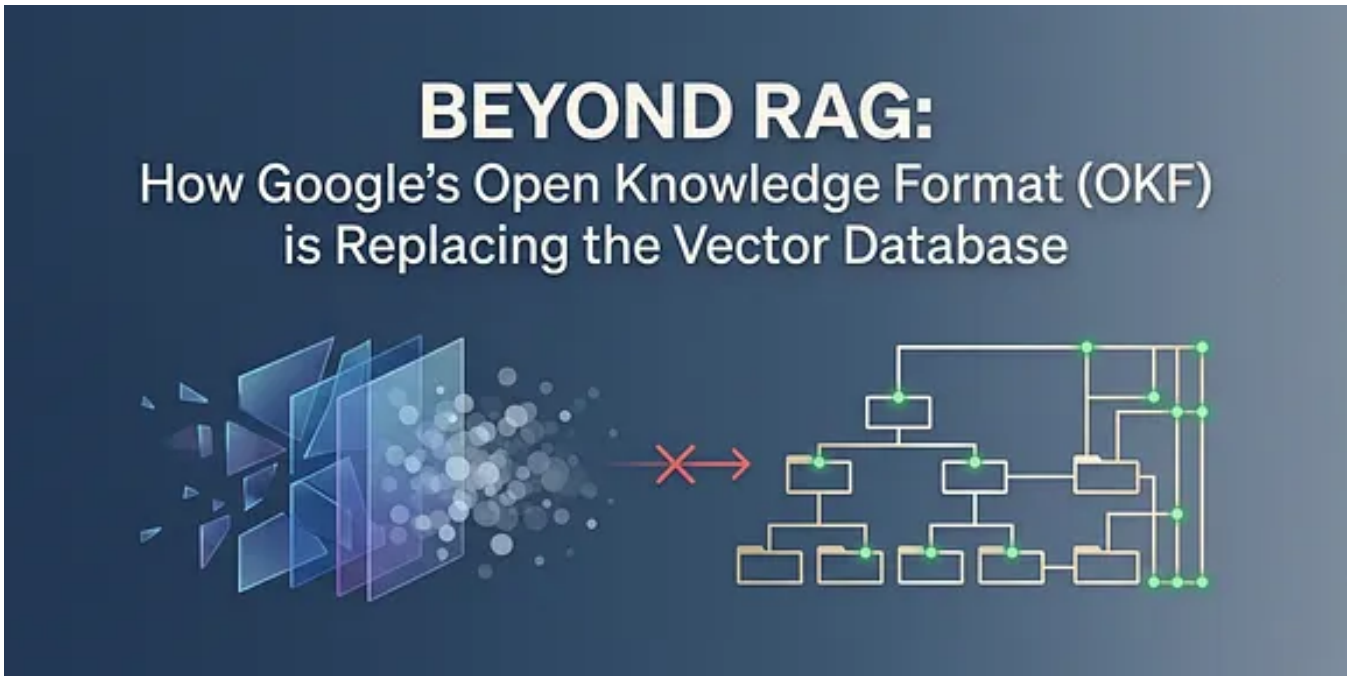


In Level Up Coding by Santosh Shinde · Jul 4

The Medallion Architecture Reconsidered: What It Solved, and Where It's Cracking

Rethinking Bronze, Silver, and Gold for the AI era

153 4 1



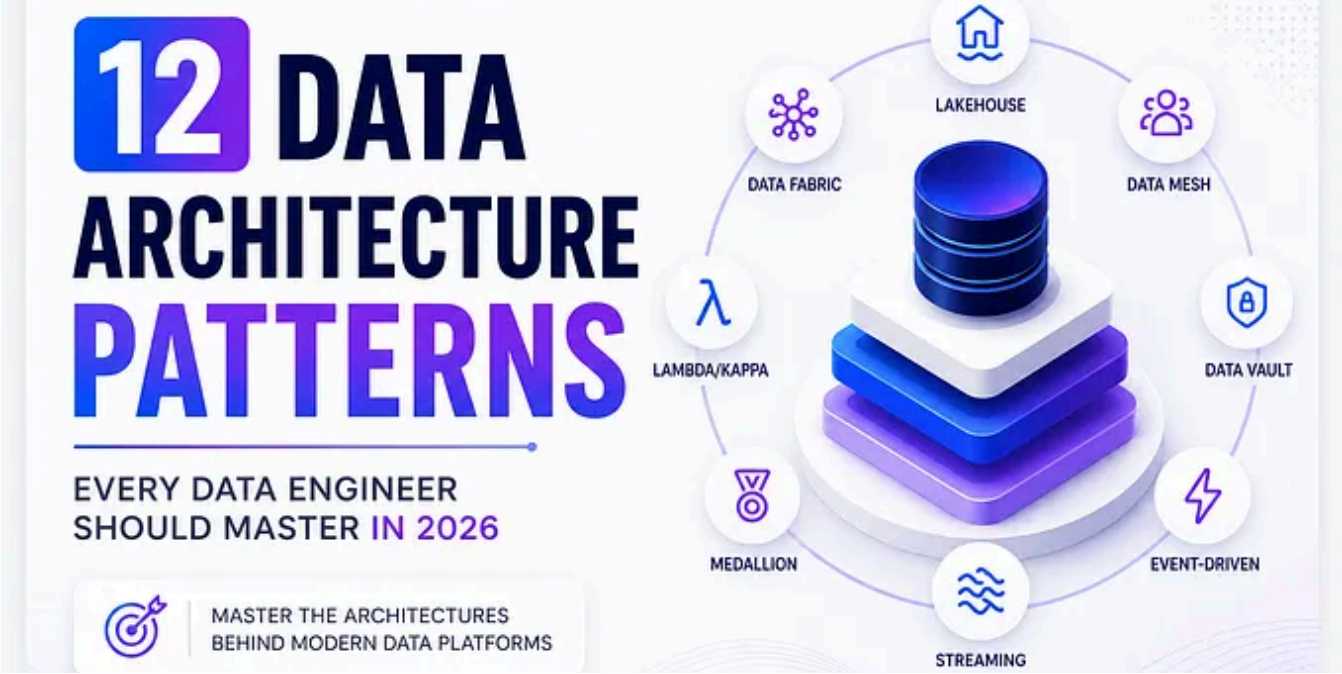
In The Code Frontier by Secret Dev · Jul 3

Beyond RAG: How Google's Open Knowledge Format (OKF) is Replacing the Vector Database

For the last t[illegible] ext problem was automa

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

1.93K



In Data Science Collective by Rohan Mistry · Jun 23

The 12 Data Architecture Patterns Every Data Engineer Should Master in 2026

From Lakehouse and Medallion to Data Mesh and Event-Driven systems — understand the architectures behind modern data platforms.

810 7 13



In UX Planet by Nick Babich · Apr 6

MCP is D

Why you sh

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

5.5K 286 178



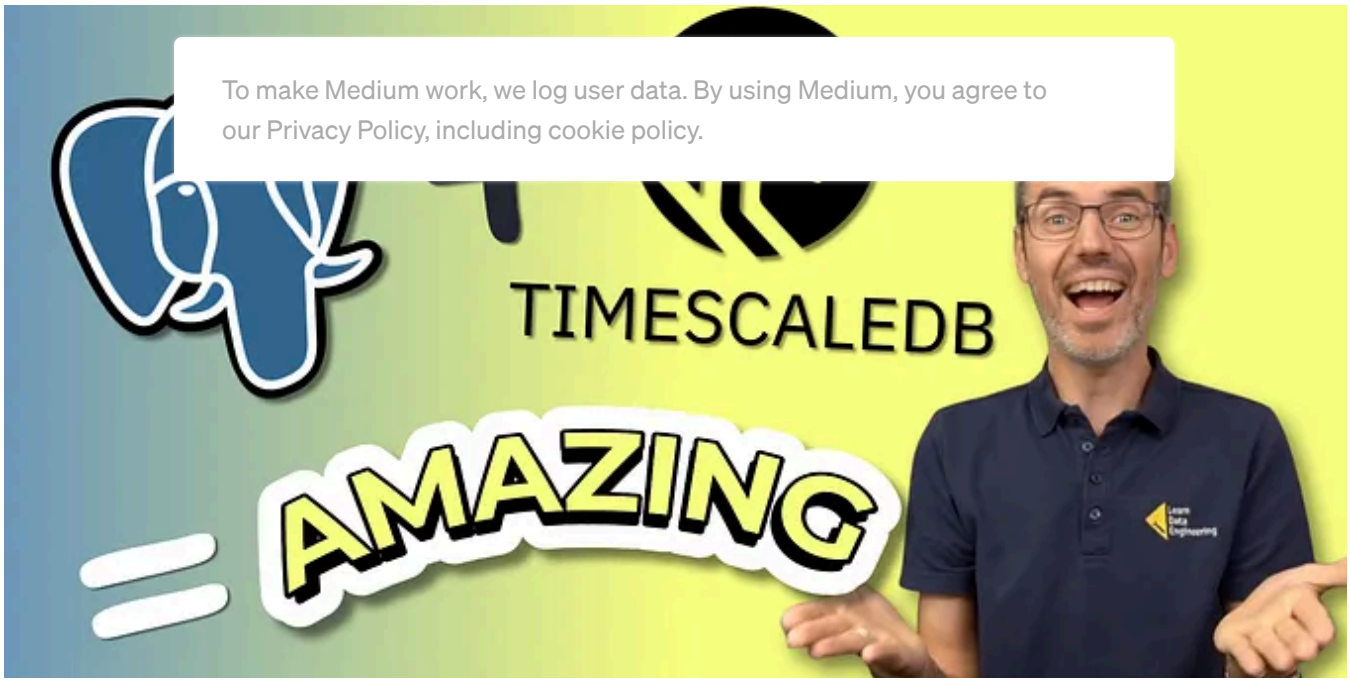
Michal Malewicz · Jul 7

You’re not going to like what AI becomes after July 12th.

This is your boiling the frog moment.

8.1K 284 72

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



In Plumbers Of Data Science by Andreas Kretz · Mar 23

Why Time-Series Data Needs a Time-Series Database

When in doubt: Postgres?

5

See more recommendations